

GAUTAM BUDDHA UNIVERSITY



SCHOOL OF ICT

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DETAILED SYLLABI (SEMESTER III-IV)

M.TECH. (COMPUTER SCIENCE AND ENGINEERING)

SPECIALIZATION IN

SOFTWARE ENGINEERING

ACADEMIC BATCH: 2025-2027

SEMESTER III



Cloud and Edge Computing			
Course Code:	CS621	Course Credits:	3
Course Category:	PCC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Explain the principles and terminology of the foundations of cloud and edge computing.			
2. Analyse problems and trade-offs concerning cloud architecture and virtualization.			
3. Apply suitable methods drawn from distributed data and compute.			
4. Design solutions that integrate edge-cloud continuum.			
5. Evaluate approaches and responsible practice relating to security, reliability and operations.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the concepts and terminology of the foundations of Cloud and Edge Computing			
2. Analyse problems using the methods covered under cloud architecture and virtualization.			
3. Apply appropriate techniques from distributed data and compute to realistic datasets or systems.			
4. Design a solution that integrates concepts from the edge-cloud continuum.			
5. Evaluate alternative approaches using technical, performance and reliability criteria.			
6. Develop and communicate a responsible, reproducible solution drawing on security, reliability and operations.			

UNIT I: FOUNDATIONS OF CLOUD AND EDGE COMPUTING

Evolution from clusters and grids to cloud and edge computing; Definition and essential characteristics of cloud computing — on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service; Overview of service models (IaaS, PaaS, SaaS) and deployment models (public, private, hybrid, multi-cloud); Drivers for edge computing — latency, bandwidth, privacy and data locality; The cloud-to-edge continuum at a glance; Representative cloud and edge use cases; Benefits, limitations and cost/performance trade-offs.

UNIT II: CLOUD ARCHITECTURE AND VIRTUALIZATION

IaaS, PaaS and SaaS; Virtual machines and containers; Orchestration; Service discovery; Autoscaling; Multi-tenancy; Cloud storage; Hypervisor types; Full and para-virtualization; Hardware support; Namespaces and control groups; Container images and registries; VM-container comparison; control loops; Resource isolation and scheduling; Public, private, hybrid and multi-cloud models; Regions and availability zones; Reference architectures; Elasticity policies; Resource provisioning; Load balancing; Cloud service selection; Performance, availability and cost comparison; Worked examples and case analysis.

UNIT III: DISTRIBUTED DATA AND COMPUTE

Distributed file systems; Key-value and NoSQL stores; Partitioning and replication; MapReduce, stream processing and serverless workflows; Data locality; Shuffling and partition strategies; Distributed joins; Lineage and recomputation; Batch-stream unification; Data skew; Checkpointing;

36th BoS Date: 30-July-2026

(Handwritten signatures and initials)

Back-pressure; Execution-plan analysis; Scalability experiments; Single-writer and multi-writer replication; Leader-follower and leaderless designs; Quorum reads and writes; Conflict-free replicated data types; Consistency anomalies; Isolation levels; Compensating actions; Failure recovery; Worked examples and case analysis; Comparative evaluation of techniques.

UNIT IV: EDGE-CLOUD CONTINUUM

Fog and edge architectures; Placement and offloading; Latency, bandwidth and energy trade-offs; IoT integration; Federated edge learning; Public, private, hybrid and multi-cloud models; Regions and availability zones; Reference architectures; Elasticity policies; Resource provisioning; Cloud service selection; Performance, availability and cost comparison; Edge node taxonomy; Cloudlets and gateways; Task-placement formulations; Computation offloading; Mobility support; Intermittent connectivity; Edge caching; Privacy-preserving analytics; Latency-energy modelling; Smart-city and industrial-IoT examples; Worked examples and case analysis.

UNIT V: SECURITY, RELIABILITY AND OPERATIONS

Zero-trust basics; Identity and access; Observability; SRE indicators; Resilience patterns; Cost governance; Sustainable computing; Case studies; Availability and reliability measures; Redundancy; Health checks; Graceful degradation; Bulkheads; Recovery-point and recovery-time objectives; Backup and disaster recovery; Fault injection; Chaos experiments; Reliability-cost trade-offs; Logs, metrics and traces; Service-level indicators, objectives and error budgets; Capacity planning; Deployment strategies; Runbooks; Incident response; Root-cause analysis; Post-incident reviews; Operational dashboards; Worked examples and case analysis.

Text Books/Reference Books:

1. M. van Steen and A. S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.
2. T. Erl, R. Puttini and Z. Mahmood, Cloud Computing: Concepts, Technology & Architecture, Pearson.
3. T. Hwang, G. Fox and J. Dongarra, Distributed and Cloud Computing, Morgan Kaufmann.
4. R. Buyya and A. V. Dastjerdi (eds.), Internet of Things: Principles and Paradigms, Morgan Kaufmann.
5. C. Pahl et al., Cloud Container Technologies: a State-of-the-Art Review, IEEE Transactions on Cloud Computing.

Handwritten signature: Akanksha

Handwritten signature: P. Buyya

AI Methods for Software Engineering			
Course Code:	CS623	Course Credits:	3
Course Category:	PCC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Explain the principles and terminology of AI-enabled software engineering.			
2. Analyse problems and trade-offs concerning code intelligence.			
3. Apply suitable methods drawn from generative AI for development.			
4. Design solutions that integrate AI for testing and operations.			
5. Evaluate approaches and responsible practice relating to trustworthy adoption.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the fundamental concepts and terminology of AI-enabled software engineering.			
2. Analyse problems using the methods covered under code intelligence.			
3. Apply appropriate techniques from generative AI for development to realistic datasets or systems.			
4. Design a solution that integrates concepts from AI for testing and operations.			
5. Evaluate alternative approaches using technical, performance and reliability criteria.			
6. Develop and communicate a responsible, reproducible solution drawing on trustworthy adoption.			

UNIT I: AI-ENABLED SOFTWARE ENGINEERING

SE lifecycle data; Task formulation; Representations of code and requirements; Classical ML and deep learning foundations for software artifacts; Software-lifecycle tasks suitable for AI; Requirements, code, test and operations data; Supervised, self-supervised and generative formulations; Feature and representation choices; Dataset construction; Temporal validation; Human-in-the-loop workflows; Productivity and quality measures; Automation boundaries; Industrial adoption risks; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures.

UNIT II: CODE INTELLIGENCE

Code embeddings; Defect and vulnerability prediction; Clone detection; Code search; Program repair; Evaluation datasets and leakage; Token-, syntax-tree-, control-flow- and graph-based representations; Benchmark construction; Negative sampling; Top-k retrieval; Precision, recall, F1 and ranking measures; Temporal validation; Cross-project generalization; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises

Handwritten signatures and initials in blue ink.

UNIT III: GENERATIVE AI FOR DEVELOPMENT

Transformers and code LLMs; Prompting; Retrieval-augmented generation; Generation, review and refactoring; Agentic development workflows; Pretraining and fine-tuning objectives; Instruction tuning; Prompt patterns; Context-window management; Retrieval indexing; Grounding and citation; Decoding controls; Structured output; Tool calling; Automated and human evaluation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs.

UNIT IV: AI FOR TESTING AND OPERATIONS

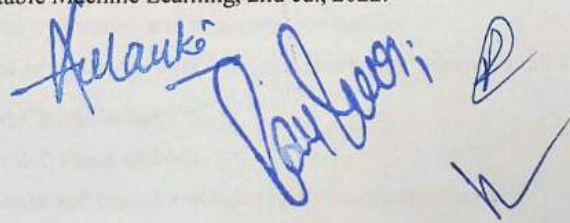
Test generation and prioritization; Log and trace analytics; Anomaly detection; Incident prediction; AIOps and human oversight; AI-based test generation; Test prioritization; Flaky-test prediction; Defect localization; Log parsing; Trace and metric correlation; Incident clustering; Root-cause assistance; Predictive maintenance of services; Alert-quality evaluation; Feedback loops; Safe automation and operator approval; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios.

UNIT V: TRUSTWORTHY ADOPTION

Quality metrics; Hallucination and security risks; Privacy and IP; Explainability; Responsible use; Empirical evaluation and industrial case studies; Data and model documentation; Robustness and distribution shift; Bias and fairness; Privacy; Intellectual-property considerations; Audit trails; Risk registers; Human approval gates; Post-deployment monitoring; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

Text / Reference Books:

1. M. Harman, S. A. Mansouri and Y. Zhang, Search-Based Software Engineering, ACM Computing Surveys.
2. C. C. Aggarwal, Machine Learning for Text, 2nd ed., Springer.
3. L. Allamanis et al., A Survey of Machine Learning for Big Code and Naturalness, ACM Computing Surveys.
4. M. Chen et al., Evaluating Large Language Models Trained on Code, arXiv, 2021.
5. C. Molnar, Interpretable Machine Learning, 2nd ed., 2022.



AI-assisted Software Engineering Lab			
Course Code:	CS681	Course Credits:	2
Course Category:	PCC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Labs (Hrs/Week):	1 (4 HRS)	Mid Sem. Exam Hours:	-
Total No. of Labs:	10	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Configure and use contemporary tools required for the laboratory domain.			
2. Implement core algorithms and system components through reproducible experiments.			
3. Measure correctness, performance, robustness and resource trade-offs.			
4. Integrate multiple components into a working domain-specific solution.			
5. Document, demonstrate and critically evaluate responsible engineering practice.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Set up and operate the required laboratory platforms and development environment.			
2. Implement and test core techniques using appropriate datasets, simulators or systems.			
3. Collect and analyse experimental evidence using suitable metrics.			
4. Diagnose failures and improve a solution through systematic experimentation.			
5. Develop an integrated mini-project with reproducible code and documentation.			

PRACTICALS

Note: Each practical shall include objective, design/algorithm, implementation, test data, observations, results and a brief critical analysis. Faculty may adapt platforms while retaining the stated learning outcomes.

Lab 1: Mining and Preparing Software Data

1. Extract commits, issues and code-review data from a repository; construct a documented dataset.
2. Build code/text representations and assess missingness, imbalance and leakage.
3. Create a reproducible experiment pipeline with versioned data and configurations.

Lab 2: Defect and Vulnerability Prediction

1. Train baseline and ensemble defect predictors.
2. Evaluate temporal splits, calibration and cost-sensitive metrics.
3. Explain predictions and inspect false positives with developers' perspective.

Lab 3: Code Search and Classification

1. Build semantic code search using embeddings.
2. Classify issues or commits and compare classical and transformer models.
3. Evaluate retrieval with precision, recall, MRR and qualitative error analysis.

Lab 4: LLM-assisted Requirements

1. Design prompts to identify ambiguity and generate acceptance criteria.
2. Ground outputs in a supplied requirements repository.
3. Measure factuality, usefulness and reviewer agreement.

36th BoS Date: 30-July-2026

Lab 5: Code Generation and Review

1. Generate a bounded function with tests using a code model.
2. Perform automated and human review for correctness, security and maintainability.
3. Compare prompt and retrieval strategies.

Lab 6: Automated Test Generation

1. Generate unit and property-based tests.
2. Measure statement, branch and mutation coverage.
3. Analyze equivalent mutants and weak oracles.

Lab 7: Program Repair and Refactoring

1. Localize faults from failing tests.
2. Generate and validate candidate patches.
3. Refactor a code smell and verify behavioural equivalence.

Lab 8: AIOps Analytics

1. Parse logs and traces; derive operational features.
2. Detect anomalies and cluster incidents.
3. Create an explainable incident summary and assess alert quality.

Lab 9: Responsible AI-SE Evaluation

1. Probe data leakage, hallucination and insecure suggestions.
2. Prepare a model/system card and mitigation plan.
3. Conduct a controlled comparison with statistical reporting.

Lab 10: Integrated Mini-project

1. Build an AI-assisted SE tool or workflow.
2. Package it with tests, documentation and reproducible evaluation.
3. Demonstrate outcomes and defend design, limitations and ethical safeguards.

Handwritten signatures:
Arun
Vijay
U

PROGRAM ELECTIVES

Bayleer;

Advanced Software Architecture and Design			
Course Code:	CS631	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Explain the principles and terminology of architectural foundations.			
2. Analyse problems and trade-offs concerning architectural styles.			
3. Apply suitable methods drawn from domain and component design.			
4. Design solutions that integrate architecture evaluation and evolution.			
5. Evaluate approaches and responsible practice relating to contemporary architectures.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the fundamental concepts and terminology of architectural foundations.			
2. Analyse problems using the methods covered under architectural styles.			
3. Apply appropriate techniques from domain and component design to realistic datasets or systems.			
4. Design a solution that integrates concepts from architecture evaluation and evolution.			
5. Evaluate alternative approaches using technical, performance and reliability criteria.			
6. Develop and communicate a responsible, reproducible solution drawing on contemporary architectures.			

UNIT I: ARCHITECTURAL FOUNDATIONS

Quality attributes; Architectural drivers; Viewpoints; Documentation; Architecture decision records; Technical debt; Stakeholder concerns; Functional and quality-attribute requirements; Scenarios and utility trees; Views and viewpoints; Component-and-connector models; Deployment views; Constraints; Risks and technical debt; Problem formulation; Terminology and assumptions; Mathematical representation; Baseline methods; Worked numerical examples; Algorithmic steps; Computational complexity; Evaluation measures; Practical limitations and application contexts; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures.

UNIT II: ARCHITECTURAL STYLES

Layered, event-driven, pipe-and-filter, microkernel, service-oriented and data-centric styles; Trade-off analysis; Stakeholder concerns; Functional and quality-attribute requirements; Scenarios and utility trees; Views and viewpoints; Component-and-connector models; Deployment views; Architecture decision records; Constraints; Risks and technical debt; Applicability, structure, benefits and liabilities of each style; Hybrid architectures; Coupling and cohesion; Control and data flow; Scalability and modifiability analysis; Style-selection exercises using enterprise case problems; Worked examples and case analysis; Comparative evaluation of techniques.

UNIT III: DOMAIN AND COMPONENT DESIGN

Domain-driven design; Bounded contexts; Components and connectors; APIs; Contracts; Dependency management; Ubiquitous language; Entities, value objects and aggregates; Context mapping; Domain events; Component responsibilities; Provided and required interfaces; Dependency inversion; Cohesion and coupling; API contracts; Package and deployment structure; Design-pattern selection; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

UNIT IV: ARCHITECTURE EVALUATION AND EVOLUTION

ATAM; Scenario-based evaluation; Fitness functions; Refactoring; Modernization; Product-line architecture; Quality-attribute scenarios; Utility trees; Architecture Tradeoff Analysis Method; Sensitivity and trade-off points; Risk themes; Architecture fitness functions; Technical-debt assessment; Evolutionary architecture; Refactoring strategies; Strangler migration; Modernization roadmaps; Architecture governance and decision review; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

UNIT V: CONTEMPORARY ARCHITECTURES

Cloud-native, reactive and serverless systems; Resilience; Observability; Security architecture; Case-study design review; Reactive manifesto; Cloud-native reference models; Function-as-a-service; Event streaming; Data-intensive architecture; Evolutionary architecture; Architecture fitness functions; Platform engineering; Modernization and migration patterns; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

Text Books/Reference Books:

1. L. Bass, P. Clements and R. Kazman, Software Architecture in Practice, 4th ed., Addison-Wesley.
2. N. Ford et al., Software Architecture: The Hard Parts, O'Reilly.
3. M. Richards and N. Ford, Fundamentals of Software Architecture, 2nd ed., O'Reilly.
4. E. Evans, Domain-Driven Design, Addison-Wesley.
5. R. N. Taylor, N. Medvidovic and E. Dashofy, Software Architecture, Wiley.

Cloud-Native and Microservices Engineering			
Course Code:	CS633	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of cloud-native principles.
2. Analyse problems and trade-offs concerning microservice design.
3. Apply suitable methods drawn from distributed data patterns.
4. Design solutions that integrate deployment and operations.
5. Evaluate approaches and responsible practice relating to resilience and governance.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of cloud-native principles.
2. Analyse problems using the methods covered under microservice design.
3. Apply appropriate techniques from distributed data patterns to realistic datasets or systems.
4. Design a solution that integrates concepts from deployment and operations.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on resilience and governance.

UNIT I: CLOUD-NATIVE PRINCIPLES

Twelve-factor applications; Containers; Immutable infrastructure; API-first design; Decomposition criteria; Cloud-native maturity; Application of the twelve-factor methodology; Stateless process design; Externalized configuration; Backing services; Disposability; Horizontal scaling; Container image layers; Immutable deployment; API-first contracts; ; Decomposition by business capability; Cloud-native readiness assessment; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

UNIT II: MICROSERVICE DESIGN

Bounded contexts; Synchronous and asynchronous communication; API gateway; Service mesh; Data ownership; Service-boundary identification; Granularity trade-offs; Synchronous REST and gRPC; Asynchronous messaging; Backend-for-frontend; Service discovery; Database-per-service; Contract versioning; Consumer-driven contract testing; Inter-service authentication; Anti-corruption layers; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

Handwritten signatures: [Signature 1], [Signature 2], [Signature 3]

36th BoS Date: 30-July-2026

UNIT III: DISTRIBUTED DATA PATTERNS

Sagas; CQRS; Event sourcing; Idempotency; Eventual consistency; Transactional outbox; Database-per-service; Saga choreography; Compensating transactions; Change-data capture; Idempotent consumers; Deduplication; Ordering guarantees; Read-model construction; Conflict handling; Schema evolution; Failure and recovery scenarios; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

UNIT IV: DEPLOYMENT AND OPERATIONS

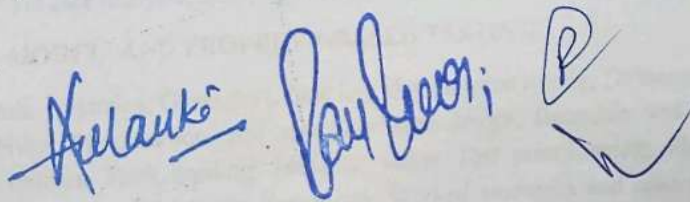
Kubernetes primitives; Configuration; CI/CD; Progressive delivery; Telemetry; SLOs; Autoscaling; Kubernetes pods, deployments, services and ingress; Configuration maps and secrets; Health probes; Resource requests and limits; Horizontal autoscaling; Rolling, blue-green and canary delivery; GitOps; CI/CD quality gates; Distributed tracing; Metrics and logs; Service-level indicators and objectives; Rollback and incident response; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios.

UNIT V: RESILIENCE AND GOVERNANCE

Timeouts, retries and circuit breakers; Chaos engineering; Security; Policy; Cost and sustainability; Migration case study and bulkheads; Health checking; Graceful degradation; Chaos testing; Service ownership; Platform standards; API governance; Compliance controls; Cost allocation; Sustainability indicators; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs.

Text Books/Reference Books:

1. S. Newman, Building Microservices, 2nd ed., O'Reilly.
2. C. Richardson, Microservices Patterns, Manning.
3. B. Burns, Designing Distributed Systems, 2nd ed., O'Reilly.
4. K. Morris, Infrastructure as Code, 2nd ed., O'Reilly.
5. B. Beyer et al., Site Reliability Engineering, O'Reilly.



Advanced Software Testing			
Course Code:	CS635	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Explain the principles and terminology of testing foundations.			
2. Analyse problems and trade-offs concerning model- and property-based testing.			
3. Apply suitable methods drawn from static and dynamic analysis.			
4. Design solutions that integrate formal verification.			
5. Evaluate approaches and responsible practice relating to verification in practice.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the fundamental concepts and terminology of testing foundations.			
2. Analyse problems using the methods covered under model- and property-based testing.			
3. Apply appropriate techniques from static and dynamic analysis to realistic datasets or systems.			
4. Design a solution that integrates concepts from formal verification.			
5. Evaluate alternative approaches using technical, performance and reliability criteria.			
6. Develop and communicate a responsible, reproducible solution drawing on verification in practice.			

UNIT I: TESTING FOUNDATIONS

Test adequacy; Coverage criteria; Mutation testing; Fault models; Combinatorial testing; Problem formulation; Terminology and assumptions; Mathematical representation; Baseline methods; Worked numerical examples; Algorithmic steps; Computational complexity; Evaluation measures; Practical limitations and application contexts; Test levels and testability; Specification- and structure-based design; Boundary and equivalence analysis; Fault seeding; Mutation score; Test prioritization; Flaky-test management; Regression selection; Test-oracle limitations; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations.

UNIT II: MODEL- AND PROPERTY-BASED TESTING

State models; Contracts; Generators; Oracles; Metamorphic testing; Differential testing; Test levels and testability; Specification- and structure-based design; Boundary and equivalence analysis; Coverage criteria; Fault seeding; Mutation score; Test prioritization; Flaky-test management; Regression selection; Test-oracle limitations; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

(Handwritten signatures)

36th BoS Date: 30-July-2026

Advanced Software Testing			
Course Code:	CS635	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of testing foundations.
2. Analyse problems and trade-offs concerning model- and property-based testing.
3. Apply suitable methods drawn from static and dynamic analysis.
4. Design solutions that integrate formal verification.
5. Evaluate approaches and responsible practice relating to verification in practice.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of testing foundations.
2. Analyse problems using the methods covered under model- and property-based testing.
3. Apply appropriate techniques from static and dynamic analysis to realistic datasets or systems.
4. Design a solution that integrates concepts from formal verification.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on verification in practice.

UNIT I: TESTING FOUNDATIONS

Test adequacy; Coverage criteria; Mutation testing; Fault models; Combinatorial testing; Problem formulation; Terminology and assumptions; Mathematical representation; Baseline methods; Worked numerical examples; Algorithmic steps; Computational complexity; Evaluation measures; Practical limitations and application contexts; Test levels and testability; Specification- and structure-based design; Boundary and equivalence analysis; Fault seeding; Mutation score; Test prioritization; Flaky-test management; Regression selection; Test-oracle limitations; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations.

UNIT II: MODEL- AND PROPERTY-BASED TESTING

State models; Contracts; Generators; Oracles; Metamorphic testing; Differential testing; Test levels and testability; Specification- and structure-based design; Boundary and equivalence analysis; Coverage criteria; Fault seeding; Mutation score; Test prioritization; Flaky-test management; Regression selection; Test-oracle limitations; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT III: STATIC AND DYNAMIC ANALYSIS

Data-flow analysis; Symbolic execution; Concolic testing; Fuzzing; Runtime verification; Control-flow graphs; Reaching definitions; Live-variable and taint analysis; Abstract interpretation; Path constraints; Symbolic states; Solver interaction; Dynamic instrumentation; Coverage-guided fuzzing; False-positive management; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations; Ethical and security implications.

UNIT IV: FORMAL VERIFICATION

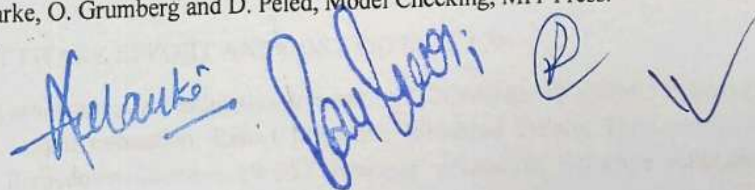
Propositional and temporal logic; Model checking; Theorem proving; SAT/SMT; Program invariants; State-transition systems; Safety and liveness; Invariants; Reachability; Temporal operators; Kripke structures; State-space explosion; Abstraction; Refinement; Counterexample analysis; Bounded and probabilistic verification; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

UNIT V: VERIFICATION IN PRACTICE

Testing concurrent and distributed systems; Safety/security cases; CI integration; Evaluation of tools and evidence; Verification plans; Assurance cases; Concurrent and distributed-system properties; Race and deadlock detection; Security-property verification; Abstraction boundaries; Tool qualification; Evidence traceability; Industrial limitations; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

Text Books/ Reference Books:

1. P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed., Cambridge.
2. G. J. Myers, C. Sandler and T. Badgett, The Art of Software Testing, 3rd ed., Wiley.
3. C. Baier and J.-P. Katoen, Principles of Model Checking, MIT Press.
4. D. Peled, Software Reliability Methods, Springer.
5. E. M. Clarke, O. Grumberg and D. Peled, Model Checking, MIT Press.



Software Measurement and Estimation			
Course Code:	CS637	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Understand the principles of software measurement and estimation.			
2. Apply software size, effort, cost, and schedule estimation techniques.			
3. Analyse software quality and productivity metrics for project management.			
4. Evaluate estimation models and risk factors in software development.			
5. Develop accurate estimation strategies for real-world software projects.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain software measurement concepts and metrics.			
2. Apply software size and effort estimation techniques.			
3. Analyse software quality, productivity, and project metrics.			
4. Evaluate estimation models for software projects.			
5. Develop estimation plans for software development projects.			
6. Assess software project performance using measurement and estimation techniques.			

UNIT I: INTRODUCTION TO SOFTWARE MEASUREMENT

Software measurement fundamentals; Goals of software measurement; Measurement theory; Scales of measurement; Direct and indirect measures; Software metrics; Product, process and project metrics; Goal Question Metric (GQM); Measurement planning; Software quality attributes; Measurement repositories; ISO/IEC software measurement standards.

UNIT II: SOFTWARE SIZE ESTIMATION

Software size estimation; Lines of Code (LOC); Function Point Analysis (FPA); Feature Points; Use Case Points; Object Points; COSMIC Function Points; Story Points in Agile; Requirement-based estimation; Early size estimation; Software complexity measures; Halstead metrics; Cyclomatic complexity.

UNIT III: SOFTWARE EFFORT AND COST ESTIMATION

Software effort estimation; Cost estimation principles; COCOMO and COCOMO II; Putnam Model; SLIM Model; Delphi estimation; Expert judgment; Wideband Delphi; Three-point estimation; PERT; Work Breakdown Structure (WBS); Resource estimation; Schedule estimation; Risk-adjusted estimation; Agile estimation techniques.

UNIT IV: SOFTWARE QUALITY AND PROJECT METRICS

Software quality metrics; Reliability metrics; Maintainability metrics; Defect density; Defect prediction; Productivity metrics; Earned Value Management (EVM); Schedule variance; Cost

Signature

36th BoS Date: 30-July-2026

variance; Software dashboards; KPI measurement; Benchmarking; Process capability; Measurement for DevOps and Agile projects.

UNIT V: ADVANCED ESTIMATION AND AI-DRIVEN SOFTWARE ANALYTICS

Machine Learning for software estimation; AI-based effort prediction; Predictive analytics for software projects; Estimation using historical repositories; Software analytics; Mining software repositories; Technical debt measurement; Risk analytics; Estimation tools; Continuous measurement; DevOps metrics; Future trends in software measurement and estimation; Industry case studies.

Text Books/ Reference Books:

1. Norman E. Fenton and James Bieman, Software Metrics: A Rigorous and Practical Approach, CRC Press.
2. Capers Jones, Applied Software Measurement, McGraw-Hill.
3. Steve McConnell, Software Estimation: Demystifying the Black Art, Microsoft Press.
4. Barry W. Boehm et al., Software Cost Estimation with COCOMO II, Pearson.
5. Roger S. Pressman and Bruce Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill.
6. Ian Sommerville, Software Engineering, Pearson.
7. Watts S. Humphrey, Managing the Software Process, Addison-Wesley.

Handwritten signatures and initials:
 Helankar, Jayaram, P, H

Secure Software Engineering and DevSecOps			
Course Code:	CS639	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3
COURSE OBJECTIVES			
1. Explain the principles and terminology of the secure development lifecycle.			
2. Analyse problems and trade-offs concerning threat modelling and design.			
3. Apply suitable methods drawn from secure implementation and testing.			
4. Design solutions that integrate DevSecOps pipeline.			
5. Evaluate approaches and responsible practice relating to operations and assurance.			
COURSE OUTCOMES			
At the end of the course, the students should be able to:			
1. Explain the fundamental concepts and terminology of the secure development lifecycle.			
2. Analyse problems using the methods covered under threat modelling and design.			
3. Apply appropriate techniques from secure implementation and testing to realistic datasets or systems.			
4. Design a solution that integrates concepts from the DevSecOps pipeline.			
5. Evaluate alternative approaches using technical, performance and reliability criteria.			
6. Develop and communicate a responsible, reproducible solution drawing on operations and assurance.			

UNIT I: SECURE DEVELOPMENT LIFECYCLE

Threat landscape; Security requirements; Misuse cases; Risk assessment; Secure design principles; Security governance; Asset classification; Abuse and misuse cases; Risk rating; Secure design review; Coding standards; Verification gates; Release approval; Vulnerability disclosure and response; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT II: THREAT MODELLING AND DESIGN

Assets and trust boundaries; STRIDE; Attack trees; Secure architecture patterns; Privacy by design; System decomposition; Data-flow diagrams; Attacker goals; STRIDE and related taxonomies; Risk prioritization; Mitigations; Residual risk; Threat-model maintenance; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations; Ethical and security implications.

UNIT III: SECURE IMPLEMENTATION AND TESTING

Common weaknesses; Secure coding; SAST, DAST, IAST; Dependency and secret scanning; Fuzzing; Input validation; Output encoding; Authentication and session controls; Access-control

36th BoS Date: 30-July-2026

Bardeen; Kelauko

R

checks; Secure error handling; Cryptographic API use; SAST, DAST and IAST; Dependency scanning; Secret detection; Fuzz testing; Security unit tests; Vulnerability triage; Remediation verification; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

UNIT IV: DevSecOps PIPELINE

Policy as code; Software composition analysis; SBOM; Artifact signing; Supply-chain security; Container and IaC scanning; Shift-left and continuous assurance; SAST, DAST, IAST and SCA placement; Secrets detection; SBOM generation; Signed artefacts; Provenance; Container and infrastructure scanning; Security gates and exceptions; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs.

UNIT V: OPERATIONS AND ASSURANCE

Zero trust; Monitoring and incident response; Vulnerability management; Compliance evidence; Maturity metrics; Case studies; Runtime protection; Identity and access review; Security telemetry; SIEM integration; Vulnerability and patch management; Incident detection and response; Forensic readiness; Penetration testing; Red-team exercises; Audit trails; Security metrics; Post-incident improvement; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

Text Books/ Reference Books:

1. G. McGraw, Software Security: Building Security In, Addison-Wesley.
2. A. Shostack, Threat Modeling, Wiley.
3. M. Howard and S. Lipner, The Security Development Lifecycle, Microsoft Press.
4. NIST, Secure Software Development Framework (SSDF), SP 800-218.
5. OWASP, Application Security Verification Standard and Testing Guide.

Handwritten signatures:
Helanki
Pavleen
[Signature]

Empirical Software Engineering			
Course Code:	CS641	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of research foundations.
2. Analyse problems and trade-offs concerning quantitative study design.
3. Apply suitable methods drawn from qualitative and mixed methods.
4. Design solutions that integrate analysis and reporting.
5. Evaluate approaches and responsible practice relating to evidence synthesis.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of research foundations.
2. Analyse problems using the methods covered under quantitative study design.
3. Apply appropriate techniques from qualitative and mixed methods to realistic datasets or systems.
4. Design a solution that integrates concepts from analysis and reporting.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on evidence synthesis.

UNIT I: RESEARCH FOUNDATIONS

Empirical questions; Theory and constructs; Validity; Ethics; Reproducibility; Open science; Problem formulation; Terminology and assumptions; Mathematical representation; Baseline methods; Worked numerical examples; Algorithmic steps; Computational complexity; Evaluation measures; Practical limitations and application contexts; Research problem, questions and hypotheses; Constructs and operational definitions; Literature positioning; Units of analysis; Sampling frame; Instrumentation; Threats to validity; Ethics approval; Preregistration; Replication package; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations.

UNIT II: QUANTITATIVE STUDY DESIGN

Controlled experiments; Quasi-experiments; Surveys; Sampling; Measurement; Power and effect size; Descriptive measures; Probability distributions; Confidence intervals; Effect sizes; Null-hypothesis tests; Multiple-comparison correction; Correlation and regression; Power analysis; Assumption checking; Robust alternatives; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations; Ethical and security implications.

Handwritten signatures and initials

36th BoS Date: 30-July-2026

UNIT III: QUALITATIVE AND MIXED METHODS

Interviews; Case studies; Grounded theory; Thematic analysis; Triangulation; Mixed-method designs; Interview and observation protocols; Purposive sampling; Transcription; Open, axial and selective coding; Memoing; Saturation; Inter-coder agreement; Member checking; Reflexivity; Reporting with evidence; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

UNIT IV: ANALYSIS AND REPORTING

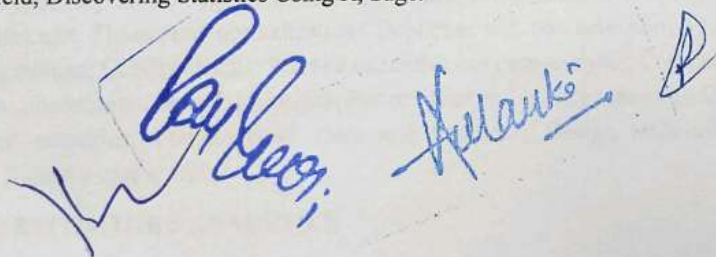
Descriptive and inferential statistics; Regression; Non-parametric tests; Qualitative coding; Uncertainty and visualization; Data cleaning; Exploratory visualisation; Assumption checking; Effect sizes and confidence intervals; Regression and non-parametric methods; Mixed-method integration; Uncertainty; Transparent tables and figures; Reproducible scripts; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT V: EVIDENCE SYNTHESIS

Systematic literature reviews; Mapping studies; Meta-analysis; Replication; Artifact evaluation; Research-paper critique; Review protocol; Database and search-string design; Inclusion and exclusion criteria; Quality appraisal; Study coding; PRISMA-style flow; Narrative synthesis; Heterogeneity; Publication bias; Research-gap formulation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

Text Books/ Reference Books:

1. C. Wohlin et al., Experimentation in Software Engineering, Springer.
2. R. K. Yin, Case Study Research and Applications, 6th ed., Sage.
3. J. W. Creswell and J. D. Creswell, Research Design, 6th ed., Sage.
4. B. Kitchenham, D. Budgen and P. Brereton, Evidence-Based Software Engineering and Systematic Reviews, CRC.
5. A. Field, Discovering Statistics Using R, Sage.



Software Engineering Analytics			
Course Code:	CS643	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of software repository ecosystems.
2. Analyse problems and trade-offs concerning data extraction and integration.
3. Apply suitable methods drawn from repository analytics.
4. Design solutions that integrate text and code mining.
5. Evaluate approaches and responsible practice relating to evaluation and deployment.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of software repository ecosystems.
2. Analyse problems using the methods covered under data extraction and integration.
3. Apply appropriate techniques from repository analytics to realistic datasets or systems.
4. Design a solution that integrates concepts from text and code mining.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on evaluation and deployment.

UNIT I: SOFTWARE REPOSITORY ECOSYSTEMS

Version control, issue trackers, code review, CI and telemetry; Research questions; Data provenance; Commit histories; Branches and merges; Pull requests; Issues; Continuous-integration logs; Releases and dependencies; API-rate limits; Bot filtering; Identity merging; Time-zone normalization; Ethical handling of developer data; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs.

UNIT II: DATA EXTRACTION AND INTEGRATION

APIs; Commit and change reconstruction; Identity resolution; Linking artifacts; Data quality and privacy; Repository APIs; Pagination and rate limits; Incremental harvesting; Schema mapping; Artefact linkage; Timestamp normalization; Duplicate and bot detection; Provenance; Privacy-preserving storage; Quality checks; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT III: REPOSITORY ANALYTICS

Descriptive metrics; Defect prediction; Ownership and socio-technical networks; Process and productivity analytics; Commit histories; Branches and merges; Pull requests; Issues; Continuous-integration logs; Releases and dependencies; API-rate limits; Bot filtering; Identity merging; Time-zone normalization; Ethical handling of developer data; Descriptive, diagnostic, predictive and

36th BoS Date: 30-July-2026

[Handwritten signature]

[Handwritten mark]

prescriptive questions; Metric validation; Temporal and project-wise splits; Class imbalance; Concept drift; Actionable visualisation; Uncertainty communication; Stakeholder-oriented interpretation; Worked examples and case analysis; Comparative evaluation of techniques.

UNIT IV: TEXT AND CODE MINING

Topic models and embeddings; Issue classification; Sentiment pitfalls; Code representations; Clone and change analysis; Tokenization and parsing; AST and graph extraction; Bag-of-words and embeddings; Topic modelling; Classification and retrieval; Issue-commit linking; Evaluation datasets; Leakage and interpretability; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT V: EVALUATION AND DEPLOYMENT

Temporal validation; Imbalance and leakage; Explainability; Dashboards; Actionable insights; Responsible analytics; Temporal and cross-project validation; Benchmark and baseline selection; Class imbalance; Leakage prevention; Calibration; Interpretability; Statistical and practical significance; Dashboard design; Stakeholder actionability; Model packaging; Batch and online inference; Drift monitoring; Retraining triggers; Privacy and governance; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices.

Text Books / Reference Books:

1. T. Menzies, L. Williams and T. Zimmermann (eds.), Perspectives on Data Science for Software Engineering, Morgan Kaufmann.
2. A. E. Hassan, The Road Ahead for Mining Software Repositories, FoSE.
3. C. Bird, T. Menzies and T. Zimmermann (eds.), The Art and Science of Analyzing Software Data, Morgan Kaufmann.
4. I. H. Witten et al., Data Mining: Practical Machine Learning Tools and Techniques, 4th ed., Morgan Kaufmann.
5. M. D. Syer et al., Replicating and Re-evaluating Software Engineering Experiments, IEEE Software.

Software Product Management and Requirements Analytics			
Course Code:	CS645	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of product strategy.
2. Analyse problems and trade-offs concerning requirements engineering.
3. Apply suitable methods drawn from product discovery and experimentation.
4. Design solutions that integrate requirements analytics.
5. Evaluate approaches and responsible practice relating to delivery and governance.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of product strategy.
2. Analyse problems using the methods covered under requirements engineering.
3. Apply appropriate techniques from product discovery and experimentation to realistic datasets or systems.
4. Design a solution that integrates concepts from requirements analytics.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on delivery and governance.

UNIT I: PRODUCT STRATEGY

Product management fundamentals; Product lifecycle; Product vision; Product strategy; Business models; Customer value proposition; Market segmentation; Competitive analysis; Stakeholder analysis; Product positioning; Opportunity assessment; Prioritization frameworks; Outcome-based roadmaps; Release planning; Product portfolio management; Product success metrics; Product lifecycle decisions; Product risk management; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary tools and practices.

UNIT II: REQUIREMENTS ENGINEERING

Requirements engineering process; Requirements elicitation; Stakeholder identification; Interviews; Workshops; Observation techniques; User stories; Use cases; Functional requirements; Non-functional requirements; Quality attributes; Acceptance criteria; Requirements prioritization; Traceability; Validation and verification; Requirement baselining; Requirement change management; Change-impact analysis; Requirements documentation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary tools and practices.

UNIT III: PRODUCT DISCOVERY AND EXPERIMENTATION

Design thinking; Customer discovery; Problem validation; Hypothesis formulation; Lean Startup; MVP design; Rapid prototyping; Wireframing; Usability testing; A/B testing; Multivariate testing; Experiment design; Randomization; Sample size estimation; Statistical significance; Guardrail metrics; Funnel analysis; Cohort analysis; Product analytics; Experiment evaluation; Data-driven decision making; Feature validation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary tools and practices.

UNIT IV: REQUIREMENTS ANALYTICS

Requirements mining; Feedback mining; Review mining; NLP for requirements; Text preprocessing; Topic modelling; Requirement classification; Requirement clustering; Duplicate requirement detection; Requirement similarity analysis; Sentiment analysis; Ambiguity detection; Requirement quality assessment; AI-based prioritization; Predictive analytics; Requirement traceability analytics; Visualization dashboards; Descriptive, diagnostic, predictive and prescriptive analytics; Metric validation; Concept drift; Stakeholder-oriented interpretation; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary AI tools and practices.

UNIT V: DELIVERY AND GOVERNANCE

Product release management; Agile product delivery; Portfolio management; OKRs; Product analytics dashboards; Product performance metrics; Customer feedback loops; Product governance; Product compliance; Accessibility; Privacy; Responsible AI considerations; Risk management; Product documentation; Product monitoring; Continuous improvement; Change governance; Regulatory compliance; Product sustainability; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Contemporary tools and practices; Industry case studies.

Text Books/Reference Books:

1. K. Pohl, Requirements Engineering, Springer.
2. S. Robertson and J. Robertson, Mastering the Requirements Process, 3rd ed., Addison-Wesley.
3. M. Cagan, Inspired, 2nd ed., Wiley.
4. T. Gilb, Competitive Engineering, Butterworth-Heinemann.
5. E. Ries, The Lean Startup, Crown Business.

Distributed Systems and Middleware			
Course Code:	CS647	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Explain the principles and terminology of models and communication.
2. Analyse problems and trade-offs concerning coordination and consensus.
3. Apply suitable methods drawn from replication and consistency.
4. Design solutions that integrate transactions and middleware.
5. Evaluate approaches and responsible practice relating to scalable systems.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the fundamental concepts and terminology of models and communication.
2. Analyse problems using the methods covered under coordination and consensus.
3. Apply appropriate techniques from replication and consistency to realistic datasets or systems.
4. Design a solution that integrates concepts from transactions and middleware.
5. Evaluate alternative approaches using technical, performance and reliability criteria.
6. Develop and communicate a responsible, reproducible solution drawing on scalable systems.

UNIT I: MODELS AND COMMUNICATION

Architectures; Processes and threads; RPC/RMI; Messaging; Naming; Time and event ordering; Remote invocation semantics; Marshalling; Request-reply protocols; Sockets; Group communication; Publish-subscribe; Message queues; Delivery guarantees; Duplicate suppression; Naming and directory services; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations; Ethical and security implications.

UNIT II: COORDINATION AND CONSENSUS

Mutual exclusion; Election; Snapshots; Consensus; Failure detectors; Byzantine faults overview; System assumptions; Safety and progress; Leader election; Replicated state machines; Paxos and Raft concepts; Quorum intersection; Network partitions; Byzantine agreement; Impossibility intuition; Consensus use cases; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies.

UNIT III: REPLICATION AND CONSISTENCY

Data-centric and client-centric consistency; Quorum protocols; Conflict resolution; CAP and PACELC; Primary-backup, multi-primary and quorum replication; Synchronous and asynchronous propagation; Read-your-writes and monotonic reads; Conflict detection and resolution; Anti-

36th BoS Date: 30-July-2026

(Handwritten signatures and initials)

entropy; Replication lag; Failover; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

UNIT IV: TRANSACTIONS AND MIDDLEWARE

Distributed transactions; Concurrency control; Commit protocols; Message brokers; Object and service middleware; Serializability; Two-phase locking; Timestamp ordering; Optimistic control; Write-ahead logging; Two-phase commit; Distributed deadlocks; Sagas; Exactly-once effects; Failure and recovery scenarios; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises; Industry case studies; Scalability considerations.

UNIT V: SCALABLE SYSTEMS

Partitioning; Caching; Stream systems; Fault tolerance; Security; Case studies of production platforms; Horizontal partitioning; Consistent hashing; Caching levels and invalidation; Load shedding; Asynchronous pipelines; Back-pressure; Hotspot and skew mitigation; Capacity modelling; Performance testing; Fault isolation; Evolutionary scaling; Worked examples and case analysis; Comparative evaluation of techniques; Implementation considerations; Performance and quality measures; Common limitations and failure scenarios; Contemporary tools and practices; Design trade-offs; Tool-supported exercises.

Text Books/Reference Books:

1. M. van Steen and A. S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.
2. T. Erl, R. Puttini and Z. Mahmood, Cloud Computing: Concepts, Technology & Architecture, Pearson.
3. T. Hwang, G. Fox and J. Dongarra, Distributed and Cloud Computing, Morgan Kaufmann.
4. R. Buyya and A. V. Dastjerdi (eds.), Internet of Things: Principles and Paradigms, Morgan Kaufmann.
5. C. Pahl et al., Cloud Container Technologies: a State-of-the-Art Review, IEEE Transactions on Cloud Computing.

Software Maintenance			
Course Code:	CS649	Course Credits:	3
Course Category:	PEC	Course (U / P):	P
Course Year (U / P):	2P	Course Semester (U / P):	3P
No. of Lectures + Tutorials (Hrs/Week):	03 + 00	Mid Sem. Exam Hours:	1.5
Total No. of Lectures (L + T):	45	End Sem. Exam Hours:	3

COURSE OBJECTIVES

1. Understand the concepts, processes, and importance of software maintenance.
2. Analyse different software maintenance techniques and reengineering approaches.
3. Apply maintenance practices for software quality improvement and evolution.
4. Evaluate software maintenance strategies, tools, and configuration management.
5. Develop effective maintenance plans for evolving software systems.

COURSE OUTCOMES

At the end of the course, the students should be able to:

1. Explain the concepts and life cycle of software maintenance.
2. Analyse software maintenance processes and maintenance issues.
3. Apply software maintenance and reengineering techniques to software systems.
4. Evaluate software quality, configuration management, and maintenance strategies.
5. Develop maintenance plans for reliable and evolving software applications.
6. Assess software quality and configuration management during maintenance.

UNIT I: INTRODUCTION TO SOFTWARE MAINTENANCE

Introduction to Software Maintenance; Software Evolution; Need and Importance of Maintenance; Types of Maintenance (Corrective, Adaptive, Perfective, Preventive); Software Maintenance Life Cycle; Maintenance Process Models; IEEE/ISO Standards for Software Maintenance; Software Documentation; Legacy Systems; Software Evolution Laws.

UNIT II: SOFTWARE MAINTENANCE TECHNIQUES

Program Comprehension; Reverse Engineering; Forward Engineering; Reengineering; Code Refactoring; Software Restructuring; Data Restructuring; Program Slicing; Software Migration; Software Modernization; Software Reuse; Component-Based Maintenance; Technical Debt.

UNIT III: SOFTWARE QUALITY AND CONFIGURATION MANAGEMENT

Software Quality in Maintenance; Software Testing during Maintenance; Regression Testing; Impact Analysis; Change Management; Software Configuration Management; Version Control Systems; Release Management; Build Management; Continuous Integration; DevOps Practices for Maintenance; Maintenance Metrics.

UNIT IV: MAINTENANCE MANAGEMENT AND AUTOMATION

Maintenance Planning; Maintenance Cost Estimation; Maintenance Team Management; Risk Management; Service Level Agreements (SLAs); Maintenance Tools; Automated Code Analysis; Static and Dynamic Analysis; AI-Assisted Code Review; Bug Tracking Systems; Automated Testing; Continuous Maintenance.

36th BoS Date: 30-July-2026

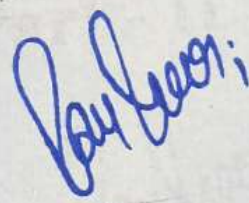
[Handwritten signatures and initials]

UNIT V: MODERN SOFTWARE MAINTENANCE PRACTICES

Cloud Application Maintenance; Microservices Maintenance; API Evolution; Containerized Application Maintenance; Security Patch Management; Performance Optimization; Predictive Maintenance using AI; Software Analytics; MLOps for Software Evolution; Green Software Maintenance; Industry Case Studies; Future Trends in Software Maintenance.

Text / Reference Books :

1. Penny Grubb and Armstrong A. Takang, Software Maintenance: Concepts and Practice, World Scientific.
2. Rajlich, V., Software Engineering: The Current Practice, CRC Press.
3. Roger S. Pressman and Bruce Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill.
4. Ian Sommerville, Software Engineering, Pearson.
5. Martin Fowler, Refactoring: Improving the Design of Existing Code, Addison-Wesley.
6. Michael Feathers, Working Effectively with Legacy Code, Prentice Hall.
7. ISO/IEC/IEEE 14764: Software Engineering – Software Life Cycle Processes – Maintenance.



SEMESTER IV

CS692: Dissertation Part - II

Barbari